

Name: Palak Chanchlani  
Class: D20A  
Roll No: 12

**BlockChain & DLT  
EXPERIMENT NO - 6**

**AIM :- Creating Smart Contracts and performing transactions using Solidity and Remix IDE**

**THEORY :-**

**1. What is a Smart Contract?**

**Answer:**

A smart contract is a **self-executing program** that runs on the blockchain. It contains rules and conditions written in code, and once those conditions are satisfied, the contract automatically performs the specified actions. Unlike traditional agreements, smart contracts do not require intermediaries such as banks or lawyers. This makes transactions faster, cheaper, and more secure. Because they are stored on the blockchain, smart contracts are **immutable** (cannot be changed once deployed) and **transparent** (anyone can verify the rules). They are widely used in areas like finance, supply chain, voting, and digital identity management, ensuring trust and efficiency in decentralized systems.

**2. How was the Smart Contract implemented in this experiment?**

**Answer:**

In this experiment, the smart contract was implemented using **Solidity**, the programming language for Ethereum. The contract was written and deployed through **Remix IDE**, which provides a user-friendly environment for coding and testing. The contract was designed to store student details such as their ID, name, course, and marks. Functions were created to add new student records and retrieve existing ones. By deploying the contract on a blockchain environment, the student data becomes **tamper-proof** and can be accessed securely. This demonstrates how smart contracts can be applied in academic systems to maintain reliable records without relying on centralized databases.

**3. What is the purpose of the addStudent function?**

**Answer:**

The **addStudent** function is responsible for **storing student information** on the blockchain. It takes inputs such as student ID, name, course, and marks, and then saves them in the contract's data structure. Since blockchain storage is permanent, once the data is added, it cannot be altered or deleted without proper authorization. This ensures that student records remain secure and trustworthy. The function highlights how blockchain can be used to maintain academic data in a decentralized manner, reducing the risk of manipulation or loss of records.

#### 4. What is the purpose of the getStudent function?

**Answer:**

The `getStudent` function is used to **retrieve student details** that have been stored in the contract. When a valid student ID is provided, the function returns the corresponding information such as name, course, and marks. This ensures that only existing records are accessed, preventing errors or invalid queries. By using this function, users can quickly verify student data directly from the blockchain. It demonstrates the **transparency and accessibility** of blockchain-based systems, where information can be securely retrieved without relying on centralized servers.

#### 5. What is the role of IPFS in this experiment?

**Answer:**

IPFS (InterPlanetary File System) plays a crucial role in handling **large files** such as certificates or documents. Storing such files directly on the blockchain would be inefficient and costly because of high gas fees and limited storage capacity. Instead, IPFS stores the actual file and generates a unique hash (address) for it. This hash is then stored in the smart contract. Whenever the file needs to be accessed, the hash is used to retrieve it from IPFS. This approach makes the system more **efficient, scalable, and secure**, as the blockchain only stores lightweight references while IPFS manages the heavy data. In the context of this experiment, IPFS ensures that student certificates can be stored and verified without burdening the blockchain.

#### Implementation & Output :-

##### Step 1: Creation of Smart Contracts

Smart contracts named **AcademicRecords.sol** and **Verification.sol** were created using Solidity in Remix IDE. These contracts are used to store student academic records and verify certificate authenticity on the blockchain.

##### **AcademicRecords.sol**

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.19;
contract AcademicRecords {

    struct Student {
        string name;
        string course;
        uint marks;
        string ipfsHash;
        bool exists;
    }
}
```

```

mapping(string => Student) public students;

function addStudent(
    string memory _id,
    string memory _name,
    string memory _course,
    uint _marks,
    string memory _ipfsHash
) public {
    students[_id] = Student(_name, _course, _marks, _ipfsHash, true);
}

function getStudent(string memory _id)
    public view
    returns (string memory, string memory, uint, string memory)
{
    require(students[_id].exists, "Student not found");
    Student memory s = students[_id];
    return (s.name, s.course, s.marks, s.ipfsHash);
}
}

```

### **Verification.sol**

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Verification {

    mapping(string => bool) public verified;

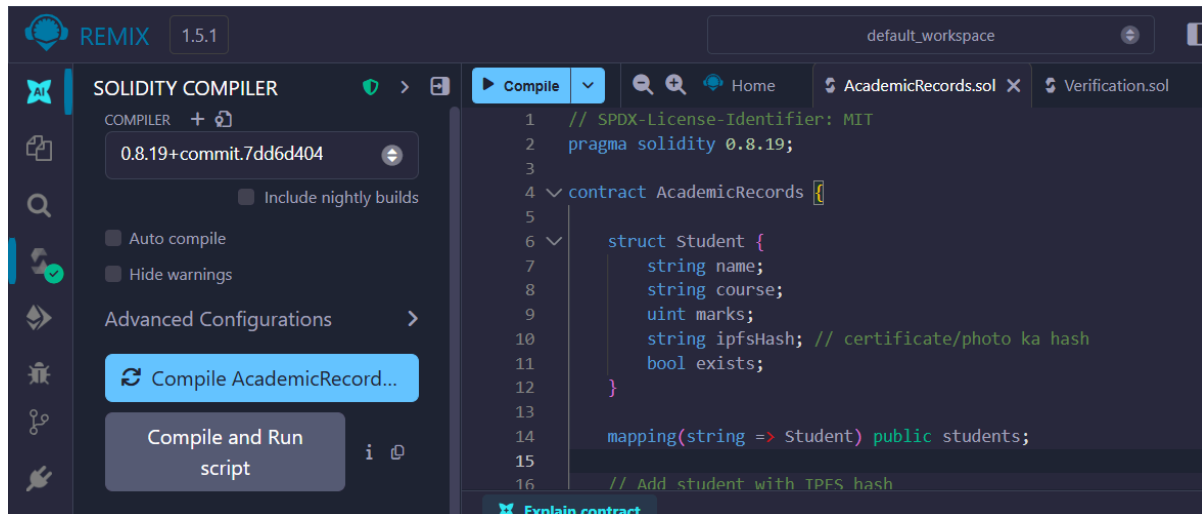
    function verifyCertificate(string memory _id) public {
        verified[_id] = true;
    }

    function checkVerification(string memory _id)
        public view
        returns (bool)
    {
        return verified[_id];
    }
}

```

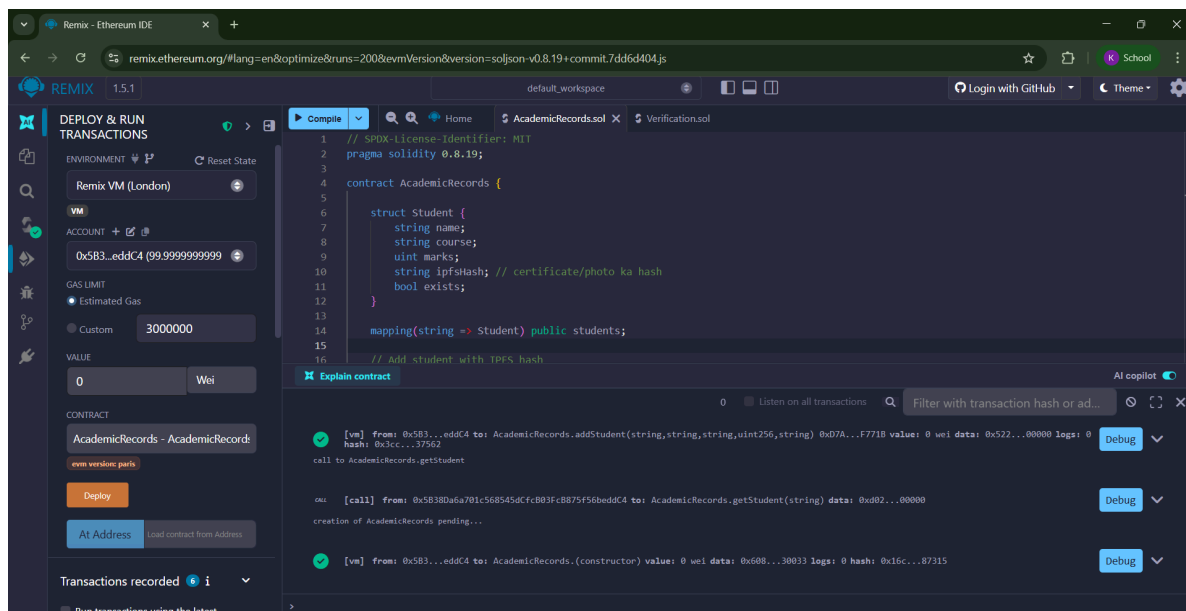
### **Step 2: Compilation of Smart Contracts**

Both smart contracts were compiled using the Solidity Compiler in Remix IDE. The compiler version 0.8.19 was selected to ensure compatibility and successful compilation.



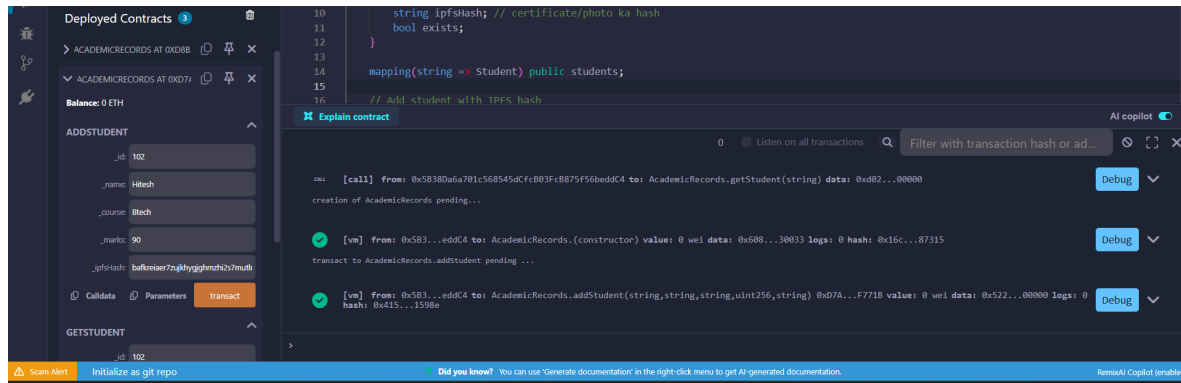
### Step 3: Deployment of Smart Contracts

The contracts were deployed using the “Deploy & Run Transactions” module in Remix IDE with the JavaScript VM environment. Successful deployment generated a contract address for interaction.



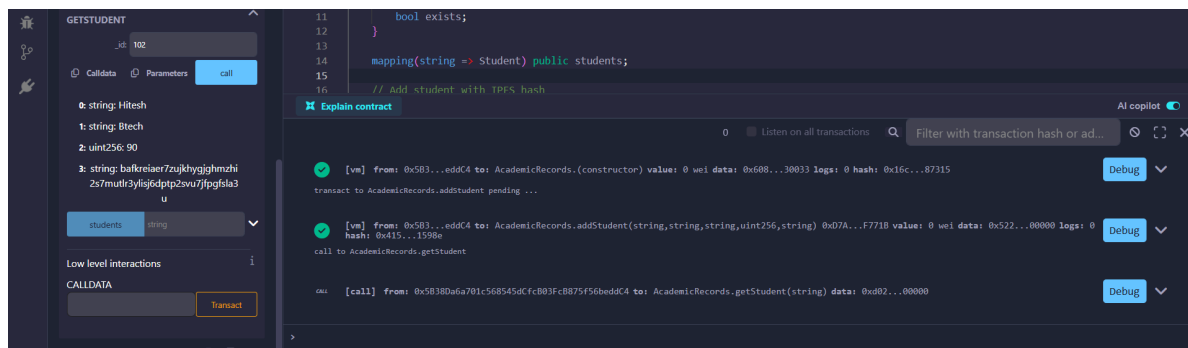
## Step 4: Execution of AcademicRecords Contract

The addStudent function was executed to store student details including ID, name, course, marks, and IPFS hash. The getStudent function was then used to retrieve and verify the stored data from the blockchain.



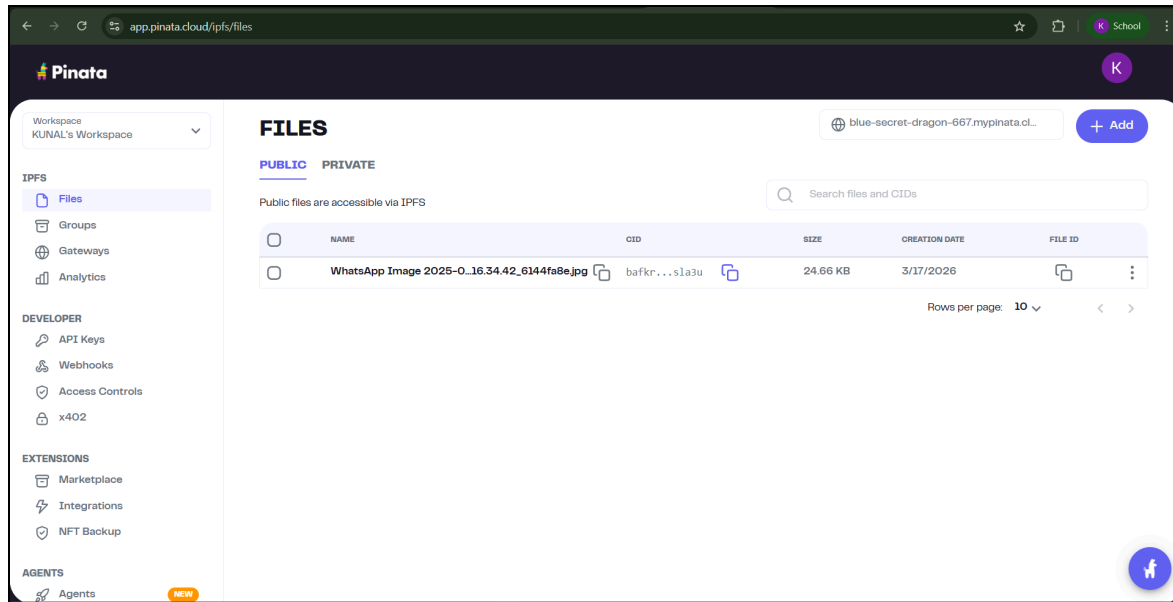
## Step 5: Execution of Verification Contract

The verifyCertificate function was used to mark a certificate as verified. The checkVerification function was used to confirm whether a particular certificate is verified or not.



## Step 6: Integration with IPFS

Student certificate images were uploaded to IPFS using a pinning service. The generated IPFS hash was stored in the smart contract, enabling secure storage and retrieval of documents without storing large files directly on the blockchain.



## Conclusion :-

This experiment helped in understanding the practical implementation of smart contracts using Solidity in Remix IDE. A smart contract was successfully designed to store and retrieve student academic records on the blockchain, ensuring data security and immutability. The system was further enhanced by integrating IPFS to store certificate images, with only the hash stored on the blockchain. This approach makes the system efficient, secure, and resistant to data tampering. Overall, the experiment demonstrated how blockchain technology can be effectively used for reliable academic credential verification.